System Card

System Overview

The final system in this repository is an ArcFace-based face verifier. Its purpose is to determine whether two face images are likely to belong to the same person. It is a pairwise verification system. It does not perform one-to-many search.

For each input pair, the system returns:

- a similarity score
- a binary verification decision
- a confidence value
- latency in CLI inference mode

The overall pipeline is as follows:

1. Read two input image paths.
2. Detect a face in each image.
3. Generate one ArcFace embedding for each detected face.
4. Normalize the embeddings.
5. Compare the two embeddings using cosine similarity.
6. Compare the resulting score against the operating threshold.
7. Return the score, decision, confidence, and latency.

If an image contains more than one face, the system selects one face for further processing. In practice, it tends to prefer the face that is largest, has a stronger detection score, and is located closer to the center of the image. If no face is detected, the system uses an all-zero embedding. This allows the pipeline to continue running, but the resulting output should be regarded as less reliable.

The frozen final system for Milestone 4 is defined by the following artifacts:

- config: `configs/arcface_best.json`
- threshold-selection artifact: `outputs/runs/arcface_sweep/arcface_sweep_summary.json`
- final fixed-threshold artifact: `outputs/runs/arcface_best/arcface_best_summary.json`
- evaluation pair set: `outputs/pairs`

Intended Use

This system is intended for:

- reproducible course-project evaluation on LFW face pairs
- command-line verification of two face images or a batch of pairs from a file
- demonstration of an embedding-based face-verification pipeline

This system is not intended for:

- one-to-many face search: This system is not intended for single input face image is compared against a gallery or database of many enrolled identities to find the closest match. It only performs pairwise verification

- Liveness detection: This system is not intended for determine whether the presented face comes from a real live person rather than a spoofing attempt, such as a printed photo, replayed video, digital screen, or mask.
- Open-set identification: This system is not intended for determining the identity of a face image by comparing it against a known set of identities. It only determines whether two images match or not, without assigning a specific identity label.

Data Summary

The final system is evaluated on the original pair set in `outputs/pairs`. The underlying images come from the Labeled Faces in the Wild (LFW) dataset, which is downloaded from TensorFlow Datasets and stored locally under `data/lfw`.

The pair files are generated deterministically. The split is performed at the identity level rather than at the image level, so the same identity does not appear across the train, validation, and test splits.

According to `outputs/pairs/manifest.json`, the dataset setup includes:

- seed: `42`
- train identities: `4599`
- validation identities: `574`
- test identities: `574`
- train images: `10646`
- validation images: `1187`
- test images: `1398`

The pair generator produces:

- `10000` train pairs
- `2000` validation pairs
- `2000` test pairs

Important data limitations include the following:

- LFW is a benchmark dataset rather than a deployment dataset.
- The dataset is collected from the web and may not reflect challenging conditions such as poor cameras, substantial blur, severe occlusion, masks, or poor cropping.
- The repository does not include reliable demographic labels for direct fairness evaluation.

For these reasons, the reported results should be interpreted as benchmark results rather than as evidence that the system will perform equally well in other settings.

Operating Threshold And Metrics

The final system version is the ArcFace fixed-threshold run saved in `outputs/runs/arcface_best/arcface_best_summary.json`.

Its main operating details are:

- embedding backend: ArcFace
- ArcFace model: `buffalo_l`
- comparison method: cosine similarity
- threshold selected from: `outputs/runs/arcface_sweep/arcface_sweep_summary.json`
- threshold selected on: validation split
- threshold selection rule: best validation F1
- final operating threshold: `0.2658909489140911`

Validation metrics at that threshold are:

- Accuracy: `0.960`
- Balanced accuracy: `0.960`
- Precision: `1.000`
- Recall: `0.920`
- F1: `0.9583`
- TP: `920`
- TN: `1000`
- FP: `0`
- FN: `80`

Final test metrics at that threshold are:

- Accuracy: `0.977`
- Balanced accuracy: `0.977`
- Precision: `1.000`
- Recall: `0.954`
- F1: `0.9765`
- TP: `954`
- TN: `1000`
- FP: `0`
- FN: `46`

These values indicate that, on the saved test set, the system produced no false positive match decisions while still missing a small number of true matches. The results are strong for this benchmark configuration, but they should not be treated as a guarantee of performance beyond this dataset and setup.

The saved run also includes confidence summaries:

- mean validation confidence: `0.1682`
- mean test confidence: `0.1654`

This confidence value is based on distance from the threshold. It is useful as a simple indicator, but it is not a calibrated probability that the prediction is

correct.

Failure Modes And Limitations

The system is more likely to fail or become less reliable under the following conditions:

- no face is detected in one or both images
- more than one face is present and the incorrect face is selected
- the same person appears very differently across the two images because of pose, lighting, expression, blur, or crop quality
- part of the face is obscured by hair, glasses, masks, or other occlusions
- the images differ substantially from the LFW benchmark style
- the same threshold is reused on a different dataset without recalibration

Other important limitations include the following:

- If no face is detected, the system uses an all-zero embedding, which reduces the trustworthiness of the result.
- The confidence score is only a margin-based measure and not a true probability estimate.
- The system does not include liveness detection or spoof detection.
- The project does not test adversarial robustness.
- The reported results are benchmark results rather than full deployment validation.

Overall, this system should be viewed as a strong academic benchmark implementation rather than a deployment-ready identity system.

Fairness-Risk Discussion

Fairness is a particularly important concern for face-verification systems because strong aggregate benchmark performance does not guarantee equitable performance across all users or conditions. A model may achieve high overall accuracy while still producing systematically different error rates for certain demographic groups or under certain imaging conditions.

In face verification, unfairness can arise in multiple ways. For example, one group may experience a higher false-positive rate, meaning individuals from that group are more likely to be incorrectly matched to someone else. Another group may experience a higher false-negative rate, meaning legitimate same-identity pairs are more likely to be rejected. In real-world deployments, both types of errors can create harm: false positives may lead to wrongful identification or unauthorized access, while false negatives may cause exclusion or denial of service.

This repository does not contain reliable demographic metadata such as age, gender presentation, skin tone, or ethnicity labels. As a result, this project does

not perform direct subgroup fairness evaluation and should not claim equal or comparable performance across populations.

Several fairness-related risks remain plausible:

- **Uneven subgroup performance**: Error rates may differ across demographic groups that were not explicitly measured in this project.
- **Image-quality-related disparities**: Performance may degrade under low lighting, blur, occlusion, or poor camera quality. These conditions may affect some populations or deployment environments more frequently than others.
- **Appearance-related confusion**: False matches may occur between visually similar but different individuals, especially in cases involving family resemblance, similar facial structure, or similar photographic conditions.
- **Condition-related false rejections**: False non-matches may occur when the same individual appears under significantly different conditions such as aging, facial hair changes, masks, makeup, pose variation, or strong lighting differences.
- **Dataset bias**: The LFW dataset consists largely of celebrity and web-sourced images and may not represent the diversity or operational conditions of real-world users.
- **Threshold transfer risk**: A threshold chosen on one benchmark dataset may produce uneven or unpredictable error rates when applied to another population or environment.

There are also misuse risks independent of measured fairness:

- deploying the system in surveillance or policing contexts
- using the system for access control or other high-stakes automated decisions
- treating benchmark-level accuracy as evidence of real-world reliability
- over-trusting the system's confidence score, which is not probability-calibrated

Because this project does not include subgroup fairness analysis, domain-specific validation, or policy-level safeguards, the most responsible interpretation is limited and explicit: this repository provides a reproducible academic benchmark implementation of a face verifier, not a fairness-validated or deployment-ready production identity system.

Operational Constraints

Reliable use depends on the following assumptions:

- Input may consist of two image paths for single inference, or a `.jsonl` or `.csv` pairs file for batch inference.
- The target face should be visible enough for the detector to identify it.
- The checked-in ArcFace configuration runs on CPU by default because `arcface_ctx_id = -1`.
- The face-detector size is configured as `640 x 640`.

- The required Python dependencies must be installed correctly.
- The image paths must exist locally, typically under `data/lfw` for the benchmark workflow.
- Docker inference assumes that the repository is mounted at `/app`.
- The first ArcFace run may take longer if the model must be downloaded.

Additional practical constraints include:

- the system is designed for pairwise verification rather than large-scale search
- the CLI is designed for reproducibility and inspection rather than production serving
- latency depends on the hardware that is used

For that reason, runtime values should be interpreted together with the Milestone 4 profiling report rather than treated as fixed operational guarantees.

Reproducibility Pointer

The final reproducible release of this project is tagged as `v1.0-final`.

Main locations:

- Project overview and commands: README.md
- Final system card: reports/system_card.md

Core commands for the final ArcFace system are:

```
python scripts/generate_pairs.py --pair-version baseline
python scripts/validate_pipeline.py --config configs/arcface_sweep.json
python -m scripts.evaluator --config configs/arcface_sweep.json
python -m scripts.evaluator --config configs/arcface_best.json
```

These commands:

- recreate the original pair set
- validate the pipeline inputs
- run threshold selection on validation
- run the final fixed-threshold ArcFace evaluation

Supporting final-system artifacts include:

- `configs/arcface_sweep.json`
- `configs/arcface_best.json`
- `outputs/pairs/manifest.json`
- `outputs/runs/arcface_sweep/arcface_sweep_summary.json`
- `outputs/runs/arcface_best/arcface_best_summary.json`
- `outputs/runs/arcface_sweep/arcface_sweep_val_roc.png`
- `outputs/runs/arcface_sweep/arcface_sweep_val_scores.jsonl`
- `outputs/runs/arcface_sweep/arcface_sweep_test_scores.jsonl`
- `outputs/runs/arcface_best/arcface_best_val_scores.jsonl`

- outputs/runs/arcface_best/arcface_best_test_scores.jsonl